Tarea Semana 4: Segundo Análisis de Casos

Código limpio, datos de texto y validación

Nombre: Britany Sofia Quiros Barahona Curso: 2026C3-G03-PROGRAMACIÓN

Fecha: 24 Octubre 2026

Este trabajo utiliza conceptos básicos de Python: input(), int(), float(), try-except, condicionales if/elif/else, operadores, strip(), lower() y f-strings.

Problema #1: Calculadora de propina inteligente

Problema

Crear un programa que calcule cuánto se debe dejar de propina y cuál será el total final de una cuenta.

Entrada

El programa solicita el monto de la cuenta y el porcentaje de propina.

Proceso

Convierte los datos a números decimales con float. Verifica que la cuenta y el porcentaje no sean negativos y que el porcentaje no supere 100. Calcula la propina con cuenta × porcentaje / 100 y suma la propina a la cuenta.

Salida esperada

Muestra la propina y el total final con dos decimales. Si se escribe un dato no numérico o fuera de los límites, muestra un mensaje de error.

Código

```
try:
    cuenta = float(input("Ingrese el total de la cuenta: "))
    porcentaje = float(input("Ingrese el porcentaje de propina: "))

    if cuenta < 0 or porcentaje < 0 or porcentaje > 100:
        print("Error: revise la cuenta y el porcentaje de propina.")
    else:
        propina = cuenta * porcentaje / 100
        total_final = cuenta + propina
        print(f"Propina: ■{propina:.2f}")
        print(f"Total final: ■{total_final:.2f}")

except ValueError:
    print("Error: debe ingresar números válidos.")
```

Ejemplo esperado: Ejemplo: cuenta ■10 000 y propina 10 % → propina ■1 000,00 y total ■11 000,00.

Problema #2: Detector de número par/impar

Problema

Crear un programa que solicite un número entero e indique si es par o impar.

Entrada

El usuario escribe un número entero.

Proceso

Convierte la entrada con int. Si el residuo de dividir el número entre 2 es cero (numero % 2 == 0), es par; de lo contrario, es impar. try-except permite manejar entradas que no sean enteros.

Salida esperada

Muestra si el número es par o impar, o un mensaje de error si la entrada no es un entero válido.

Código

```
try:
    numero = int(input("Ingrese un número entero: "))

    if numero % 2 == 0:
        print(f"El número {numero} es par.")
    else:
        print(f"El número {numero} es impar.")

except ValueError:
    print("Error: debe ingresar un número entero válido.")
```

Ejemplo esperado: Ejemplo: entrada 7 → “El número 7 es impar.”

Problema #3: Clasificador de edad

Problema

Crear un programa que clasifique la edad como menor de edad, adulto o adulto mayor.

Entrada

El usuario ingresa su edad en años.

Proceso

Convierte la entrada a entero y verifica que no sea negativa. Si es menor que 18, es menor de edad; si es menor que 65, es adulto; en caso contrario, es adulto mayor.

Salida esperada

Muestra la categoría correspondiente. Si la edad es negativa o no es un entero válido, muestra un mensaje de error.

Código

```
try:
    edad = int(input("Ingrese su edad en años: "))

    if edad < 0:
        print("Error: la edad no puede ser negativa.")
    elif edad < 18:
        print("Clasificación: menor de edad.")
    elif edad < 65:
        print("Clasificación: adulto.")
    else:
        print("Clasificación: adulto mayor.")

except ValueError:
    print("Error: debe ingresar una edad usando números enteros.")
```

Ejemplo esperado: Ejemplo: entrada 21 → “Clasificación: adulto.”

Problema #4: Validador de compra con descuento

Problema

Solicitar el nombre del cliente, el monto de la compra y si es cliente frecuente. Aplicar un descuento según las reglas definidas.

Entrada

El programa solicita nombre, monto de compra y respuesta si/no. Se aceptan “sí”, “si” y “no”, sin importar espacios al inicio/final o mayúsculas.

Proceso

Limpia el nombre con strip() y convierte la respuesta a minúsculas con lower(). Si el nombre está vacío o el monto es negativo, muestra un error. Si es cliente frecuente y compra ■50 000 o más, recibe 10 %. Si solo cumple una condición, recibe 5 %. Si no cumple ninguna, recibe 0 %. Después calcula descuento y total.

Salida esperada

Muestra el nombre, porcentaje y monto del descuento, y total a pagar. Los datos inválidos generan un mensaje claro.

Código

```
try:
    nombre = input("Ingrese el nombre del cliente: ").strip()
    monto = float(input("Ingrese el monto de la compra: ■"))
    respuesta = input("¿Es cliente frecuente? (si/no): ").strip().lower()

    if nombre == "":
        print("Error: el nombre no puede estar vacío.")
    elif monto < 0:
        print("Error: el monto no puede ser negativo.")
    elif respuesta != "si" and respuesta != "sí" and respuesta != "no":
        print("Error: responda si o no.")
    else:
        if respuesta == "si" or respuesta == "sí":
            frecuente = True
        else:
            frecuente = False

        if frecuente and monto >= 50000:
            descuento_porcentaje = 10
        elif frecuente or monto >= 50000:
            descuento_porcentaje = 5
        else:
            descuento_porcentaje = 0

        descuento = monto * descuento_porcentaje / 100
        total_pagar = monto - descuento

        print(f"Cliente: {nombre.title()}")
        print(f"Descuento aplicado: {descuento_porcentaje}% (■{descuento:.2f})")
        print(f"Total a pagar: ■{total_pagar:.2f}")

except ValueError:
    print("Error: el monto debe ser un número válido.")
```

Ejemplo esperado: Pruebas: ■60 000 y sí → 10 %; ■60 000 y no → 5 %; ■30 000 y sí → 5 %; ■30 000 y no → 0 %.

Pruebas manuales

Caso	Monto	Frecuente	Resultado esperado
1	■60 000	Sí	10 %; paga ■54 000
2	■60 000	No	5 %; paga ■57 000
3	■30 000	Sí	5 %; paga ■28 500
4	■30 000	No	0 %; paga ■30 000
5	-■100	Sí	Error por monto negativo
6	Texto	Sí	Error por dato no numérico

Conclusión

La validación más importante fue comprobar que los datos numéricos fueran válidos y que no fueran negativos cuando correspondía. Durante las pruebas, un posible error es escribir texto en lugar de un número o una respuesta distinta de “sí” o “no”. La limpieza del código con nombres claros, strip() y lower() ayuda a evitar errores por espacios o mayúsculas y hace que el programa sea más fácil de leer y mantener.